Course Project 5: Neural Network Architectures & Optimization

Objective: To gain hands-on experience designing, training, and debugging neural networks. Students will transition from a basic Multi-Layer Perceptron (MLP) to a Convolutional Neural Network (CNN), focusing on the interpretation of "Training Curves" and the application of regularization techniques discussed in the lectures.

Dataset:

- Name: Fashion-MNIST
- Description: 60,000 training images and 10,000 test images of 10 different fashion categories (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot). Each image is a 28x28 grayscale integer matrix.
- Access: Available directly via PyTorch (torchvision.datasets.FashionMNIST)

Assignment Instructions

Part 1: The Baseline MLP (Mathematical Foundation)

- Task: Implement a standard Multi-Layer Perceptron (MLP) as described in the "Intro to Neural Networks" slides.
- Architecture Requirements:
    - Input Layer: Flatten the 28x28 images into a vector of size 784.
    - Hidden Layers: Two hidden layers with 128 and 64 neurons respectively. Use ReLU activation functions.
    - Output Layer: 10 neurons (one for each class) with Softmax output.
- Training: Use Stochastic Gradient Descent (SGD) and the Cross-Entropy Loss function. Train for 20 epochs.

Part 2: The "Overfitting" Experiment & Regularization

- Task: Intentionally create an overfitting scenario and then fix it, utilizing the concepts from the "Training tips" section of the lecture.
- Step A (Force Overfitting): Create a new model with excessive capacity (e.g., 4 hidden layers with 512 neurons each) and train it on a small.subset of the data (e.g., only use 10% of the training images).

- Step B (Regularization): Apply Dropout (probability 0.5) or L2 Regularization to this large model and retrain.
- Comparison: Compare the gap between Training Loss and Validation Loss for Step A vs. Step B.

Part 3: Convolutional Neural Networks (CNNs)

- Task: Implement a LeNet-style CNN to utilize the translational variance inductive bias inherent in images.
- Architecture:
    - 2 Convolutional layers (kernel size 5x5).
    - Activation function ReLU.
    - Max Pooling layers between convolutions.
    - Flatten and connect to a dense (fully connected) output layer.
- Comparison: Compare the final accuracy of this CNN against your Part 1 MLP.

What you need to do:

We provide a Jupyter notebook code template. You will supplement the code and produce the results. You also need to provide detailed explanation on the code you supplement (by using "#" to add your explanation as comments.

There are four (4) missing parts in the template. Some detailed requirements on the architecture of MLP, NN, or CNN have been provided in the template (e.g., padding size, filter size, ...). Please follow the requirements.